1.- El paquete que se importa de forma automática y predeterminada en todos los programas de Java es java.lang.
2.- B. El paquete que no necesita ser importado explícitamente en un programa Java es java.lang
3.- Métodos estáticos (static): Al ser un método estático, sumar pertenece a la clase Utilidades en su conjunto y no a una instancia (objeto) específica de la misma. Por lo tanto, se puede invocar directamente utilizando el nombre de la clase (Utilidades.sumar(5, 10)) sin necesidad de crear un objeto con new Utilidades().
4.- Momento de ejecución: Se ejecuta exactamente una vez cuando la clase es cargada por primera vez en la Memoria por la JVM (Java Virtual Machine). Esto ocurre antes de que se cree cualquier instancia (objeto) de la clase y antes de que se invoque cualquier método estático.
5.- En Java, el modificador de acceso `protected` determina la visibilidad de un campo, método o constructor. Se puede acceder a él desde:

- La misma clase donde está declarado.
- Cualquier clase dentro del mismo paquete (similar a la visibilidad *package-private* o por defecto).
- Subclases (clases hijas), incluso si dichas subclases se encuentran en un paquete completamente diferente.

6.- **`public`**: Es un modificador de acceso que puede preceder a la palabra `class` en clases de nivel superior (top-level) y en clases anidadas para indicar que la clase es accesible desde cualquier otra clase o paquete.
`static`: Puede preceder a la palabra `class` únicamente cuando se trata de una **clase anidada (nested class)** o miembro estático, indicando que dicha clase interna no requiere una instancia de la clase externa para ser utilizada.
7.- Una clase de nivel superior (top-level class) es aquella que se declara directamente en un paquete y no está anidada dentro de otra clase.
8.- **El problema:** El método `one()` está declarado como `private` dentro de la clase `X`. En Java, los miembros privados (`private`) solo son visibles y accesibles dentro de la misma clase donde se declaran. Las subclases (como `Y`) **no heredan** ni pueden acceder directamente a los miembros privados de su superclase. Por lo tanto, cuando `two()` intenta llamar a `one()`, el compilador genera un error indicando que `one()` tiene acceso privado en `X`.
La solución: Al cambiar el modificador de `one()` a `protected` (o bien a `public` o dejarlo con visibilidad de paquete), el método se vuelve accesible para la subclase `Y`, permitiendo que el código compile y se ejecute correctamente.
9.- Llamada al constructor privado: Aunque el constructor de ExtendedA está declarado como private, se puede invocar sin problemas desde el método main porque ambos se encuentran dentro de la misma clase (ExtendedA). Los miembros y constructores privados son totalmente accesibles dentro de la clase donde fueron declarados. las opciones c, d y e son falsas porque en Java los constructores sí pueden utilizar modificadores de acceso como private, protected o public según se requiera (por ejemplo, para patrones de diseño como Singleton).

10.- **Carga de la clase (Fase Estática):**

- Cuando la JVM carga la clase `Prioridad` para ejecutar el método `main`, se ejecutan los bloques estáticos y la inicialización de variables estáticas en el orden en que aparecen.
- El bloque `static { System.out.println("static"); }` se ejecuta primero, imprimiendo **`static`**.

2. **Creación de la instancia (`new Prioridad()`):**
   - Cuando se instancia el objeto, antes de que se ejecute el constructor, se ejecutan los **bloques de inicialización de instancia** (los bloques entre llaves `{}` sin la palabra `static`).
   - El bloque `{ System.out.println("default"); }` se ejecuta e imprime **`default`**.
3. **Ejecución del constructor:**
   - Después del bloque de instancia, se ejecuta el cuerpo del constructor: `System.out.println("Constructor");`, imprimiendo **`Constructor`**.
4. **Llamada al método (`p.getSaludo()`):**
   - Finalmente, se invoca el método `getSaludo()`, el cual imprime el contenido de la variable estática `saludo`, es decir, **`Variable`**.

Secuencia completa de salida: **static -> default -> Constructor -> Variable**.

11.- **Carga de la clase (Fase Estática):**

- Antes de que la JVM pueda ejecutar el método `main`, debe cargar la clase `StaticNonStaticBlockTest` en la memoria.
- En este preciso instante (antes de cualquier llamada a métodos o creación de objetos), se ejecuta el bloque estático: imprimiendo **`Static block`**.

2. **Ejecución del método `main`:**
   - La JVM ejecuta el método `main`.
   - La primera instrucción dentro de `main` es `StaticNonStaticBlockTest.staticMethod();`, lo que imprime **`Static method`**.
3. **Instanciación del objeto (`new StaticNonStaticBlockTest()`):**
   - La segunda instrucción de `main` crea un nuevo objeto de la clase.
   - Al instanciar, primero se ejecutan los **bloques de inicialización de instancia** (no estáticos): imprimiendo **`Instance block`**.
   - Inmediatamente después, se ejecuta el **constructor** de la clase: imprimiendo **`Constructor`**.

Secuencia final: **Static block -> Static method -> Instance block -> Constructor**.

12.- **Impresión (`System.out.println(m.value);`):**

- Como `m` apunta al segundo objeto creado, al imprimir su propiedad `value` se muestra **`ABCX`**.

13.- En Java, el orden de ejecución es el siguiente:

1. Bloques estáticos (`static`): Se ejecutan una sola vez, cuando se inicializa la clase, siguiendo el orden en que aparecen en el código.
2. Bloques de instancia: Se ejecutan cada vez que se crea un objeto, en el orden en que están declarados.
3. Constructor: Se ejecuta después de los bloques de instancia.